

output for our label. Instead we'll use one hot encoding.

1. Add functions to convert between class name and class index:

```
a. function getClassIndex(className) {
b.     if (className === 1 || className === "1") {
c.         return 0; // 1 bedroom
d.     }
e.     else if (className === 2 || className === "2") {
f.         return 1; // 2 bedrooms
g.     }
h.     else {
i.         return 2; // 3 bedrooms
j.     }
k. }
l.
m. function getClassIndex(classIndex) {
n.     if (classIndex === 2)
o.         return "3+";
p.     else
q.         return classIndex+1;
r. }
```

2. Replace the code to extract the label. This will create an index for the class, then encode each index as a size 3 1D tensor according to one hot encoding.

- // Extract Labels (outputs)
- `const` labelValues = points.map(p => getClassIndex(p.class));
- `const` labelTensor = tf.tidy(() => tf.oneHot(tf.tensor1d(labelValues, 'int32'), 3));

Create a multi-class classification model

1. In `createModel`, change the `loss` function to `categoricalCrossentropy`.
2. Change the activation functions to `softmax`.
3. Change the final/output layer to have three units.
4. Change `storageID` to `kc-house-price-multi` so that we will not overwrite the previous model on save:

a. `const` storageID = `kc-house-price-multi`;

Update the heatmaps

1. Change the final part of `plotPredictionHeatmap` to split or "unstack" the tensor, one for each label, and display each as a heatmap:

- a. `tf.unstack(values, 2).forEach((values, i) => {`
- b. `const data = {`
- c. `values,`
- d. `xTickLabels,`
- e. `yTickLabels,`
- f. `});`
- g.
- h. `tfvis.render.heatmap({`
- i. `name: `Bedrooms: ${getClassName(i)} (local)`,`
- j. `tab: "Predictions"`
- k. `}, data, {height: size});`
- l. `tfvis.render.heatmap({`
- m. `name: `Bedrooms: ${getClassName(i)} (full`
- n. `domain)`,`
- o. `tab: "Predictions"`
- p. `}, data, {height: size, domain: [0, 1]});`

Making predictions

1. Update the predict function to handle multiple outputs:

- `const` outputs = outputTensor.dataSync();
- `let` outputString = "";
- `for` (`let` i = 0; i < 3; i++) {
- outputString += `Likelihood of having
\${getClassName(i)} bedrooms is:
\${(100*outputs[i]).toFixed(1)}%
`;
- }
- document.getElementById("prediction-
output").innerHTML = outputString;